



Prompt迭代管理规范RFC

🕒 创建时间	@October 23, 2025 2:38 PM
⚙️ 状态	无
📋 目标	
🕒 最后更新于	@November 12, 2025 9:46 AM
📋 训练数据集	
📋 数据影响范围	
☑️ 排除训练	☐
➤ Parent item	<u>Prompt 管理</u>

目标

All In One

发布流程

开发迭代及合并流程

多人开发场景示例

只有一个人在开发一个需求

我要在其他正在开发的内容基础上做修改

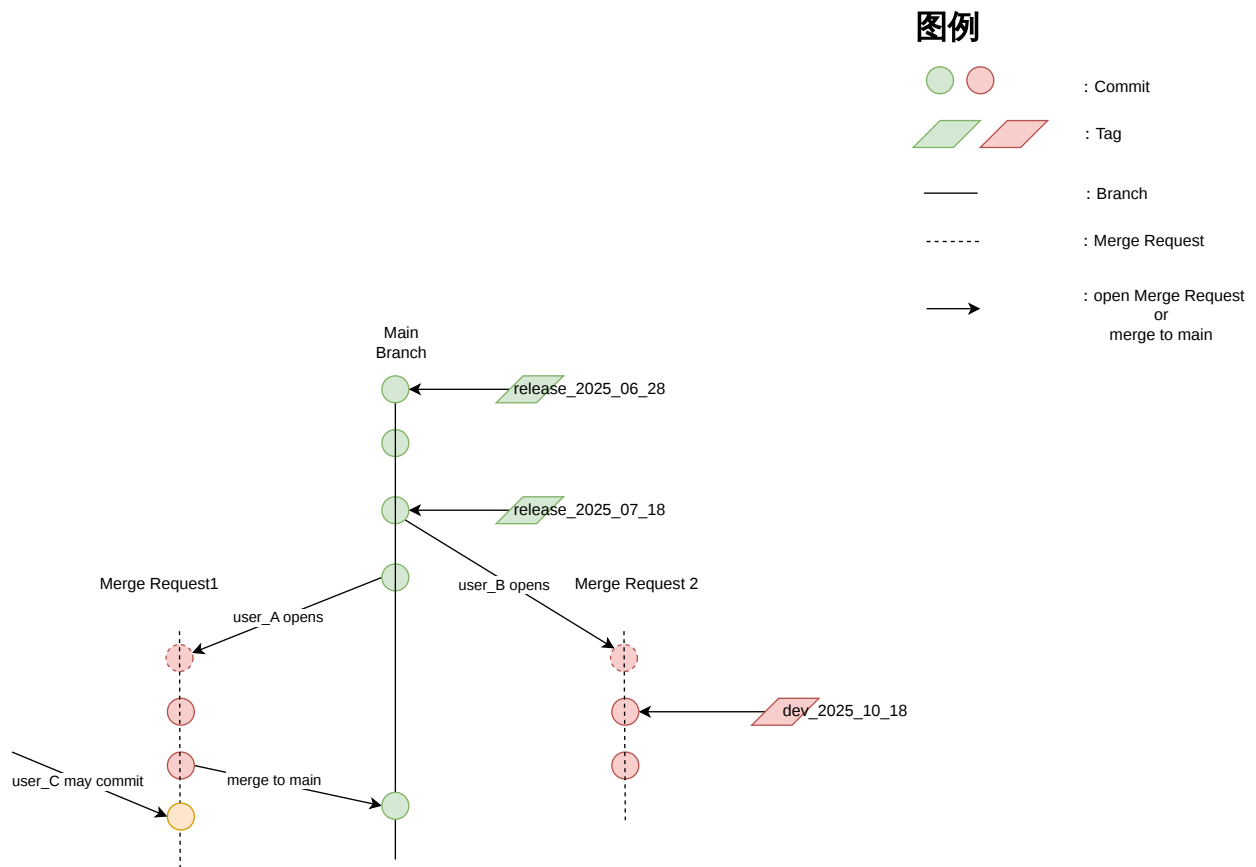
关于Review

一个Assistant的结构

目标

1. 定义清楚发布、可被线上使用的操作流程
2. 定义清楚System Prompt及User Prompt Template的迭代及合并流程
3. 定义一个Assistant的结构

All In One



发布流程

- 正式发布以main分支上的commit为发布对象，具体实现方式是在main分支的commit上打上 `release-{date}` 标签。代码中则实现将当前最新的一个 `release-{date}`，push到应用方。
 - 只有在main分支上的内容可以在线上运行!!!**
- 测试发布支持以其他分支的commit作为发布对象，具体实现方式是在任意分支(如 merge request)的commit上打上 `dev-{date}` 标签。作为测试则可以直接手动选择要使用那个 `dev-{date}` 对应的commit。
 - 非main分支上的内容只能在测试环境运行!!!

开发迭代及合并流程

flowchart LR

```
A[开始/领取需求] --> B[在对应的 MR 上开发，可能多次修改]
B --> D{是否需要发布?}
D -- 是 --> E[Review]
E -- Review通过 --> F[合并 MR 到 main 分支]
F --> G[MR 被删除（合并即关闭）]
G --> H[结束]
```

修改的单元是一个merge request，一个次迭代上线前的所有变更都在一个merge request中管理。一个merge request在被merge到main分支时，即被删除。

多人开发场景示例

只有一个人在开发一个需求

此时，开发的同学只需要保证自己对于该需求的改动都提交到了对应的merge request。在需要发布时，将merge request合并到main分支。

我要在其他正在开发的内容基础上做修改

在这我们假设human1在mr1(当前的head是commit1)上开发，而human2想在mr1的基础上修改。

此时，有两种做法：

1. 以mr1为基础开一个mr2

▼ 此时，有一个场景会有问题，即human1又继续在mr1上做了提交，如果想同步就需要做rebase操作或者手动同步的操作。

flowchart TD

```
A[mr1 @ commit1] --> B[从 mr1 新建 mr2]
B --> C[human2 在 mr2 开发/提交]
A --> D[human1 在 mr1 继续提交 -> commit2/commit3...]
C --> E{mr1 是否有新增提交?}
```

```
D --> E
E -- 否 --> F[继续在 mr2 开发]
E -- 是 --> G[同步操作]
G --> H[方案一：rebase mr2 到最新的 mr1]
G --> I[方案二：手动同步<br/>如 cherry-pick / merge /
手动拣选变更]
H --> J[解决冲突后继续在 mr2 开发]
I --> J
```

2. 直接在mr1上修改（不推荐）

- a. 如果修改不大，可以直接选择在mr1上继续提交做修改。

关于Review

1. 每个assistant会至少分配2个owner(原则上产品、研发、教研角色尽量包含)
2. 每次assistant上的修改要获得2个以上assistant owner的通过
3. Review的周期频率：
 - a. 在合并进入main分支前必须Review
 - b. 如有变更，至少在每周的粒度上做Review
4. Review包含的素材：
 - a. Prompt本身的变更
 - b. 评测报告

一个Assistant的结构

assistant-name/	# 如（探索一对一讲解）
├ system_prompt.txt	
├ first_user_prompt	# 首轮user_prompt的模版
│ └ template.txt	
│ └ samples	# first user prompt templat

e的填充参数示例文件夹

```
| | └─ sample1.txt
| | └─ ...
└─ possible_user_prompt_template # 非首轮可能使用的user_prompt
的模版列表
| └─ template1.txt
| └─ ...
└─ llm_configs.json # 模型配置
└─ single_turn_test_cases/ # 文件夹（固定历史的单轮测试）
    └─ testcase1.jsonl
    └─ ...
    └─ metric.py
└─ simulated_chat_test_cases/ # 文件夹（模拟学生的人设文件夹）
    └─ sim_profile1.txt
    └─ ...
    └─ metric.py
```

用 DB 存储和 git 存储方案对比

如何开发和调试 system prompt

Prompt 收口流程